BTF Viewer

Newbie workflow for RTOS trace diagnosis

Find evidence first → scope the phase → measure → verify on timeline → then ask AI to explain.

Documentation Map

Document	Purpose
`README.md`	Product overview, quick start, UI, settings, export, demo
`WORKFLOWS.md`	Step-by-step diagnosis procedure
`STATISTICS.md`	Metric definitions, formulas, charts
`AI.md`	AI tools, planner, models, validator

> `README → WORKFLOWS → STATISTICS → AI`

The Core Rule

```
Open trace
  ↓
Check health
  ↓
Scope phase
  ↓
Read Analysis findings
  ↓
Open named Statistics section
  ↓
Click to timeline evidence
  ↓
Confirm / reject
  ↓
AI explains after evidence exists
  ↓
Compare after the fix
```

> AI is an explanation layer. Statistics and the timeline are the evidence layer.

10-Minute Triage

Step	Action
1	Open trace, press `Ctrl+0`, enable **Load**
2	Open **Statistics** + toolbar **Analysis**
3	Check **Trace Health (TICK)** first
4	Open only Statistics sections named by Findings
5	Click **Max** / **p95** / row / chart point
6	Place C1–Cn cursors + **Limit to C1–Cn**
7	Re-check findings inside the scoped phase
8	Use AI only after the timeline agrees
9	Export HTML or run **Trace Compare**

The Ladder

Use this order for most issues

Top-Down Analysis Ladder

```
① Health       Trace Health (TICK)
② Scope        Cursors + Limit to C1–Cn
③ Balance      Core Utilisation → Breakdown / Concurrent / Switch
④ CPU / WCET   Top Tasks → Execution Time Per Slice
⑤ Latency      Blocking → Dispatch → Preemption
⑥ Concurrency  Migrations → Heatmap / Chord → Mutex → Priority Inheritance
⑦ Compliance   Affinity → Lifecycle → Deadlines → Tags / Intervals
⑧ Compare      Trace Compare before / after
⑨ Explain      AI investigation / report
```

Stop when the evidence explains the symptom.

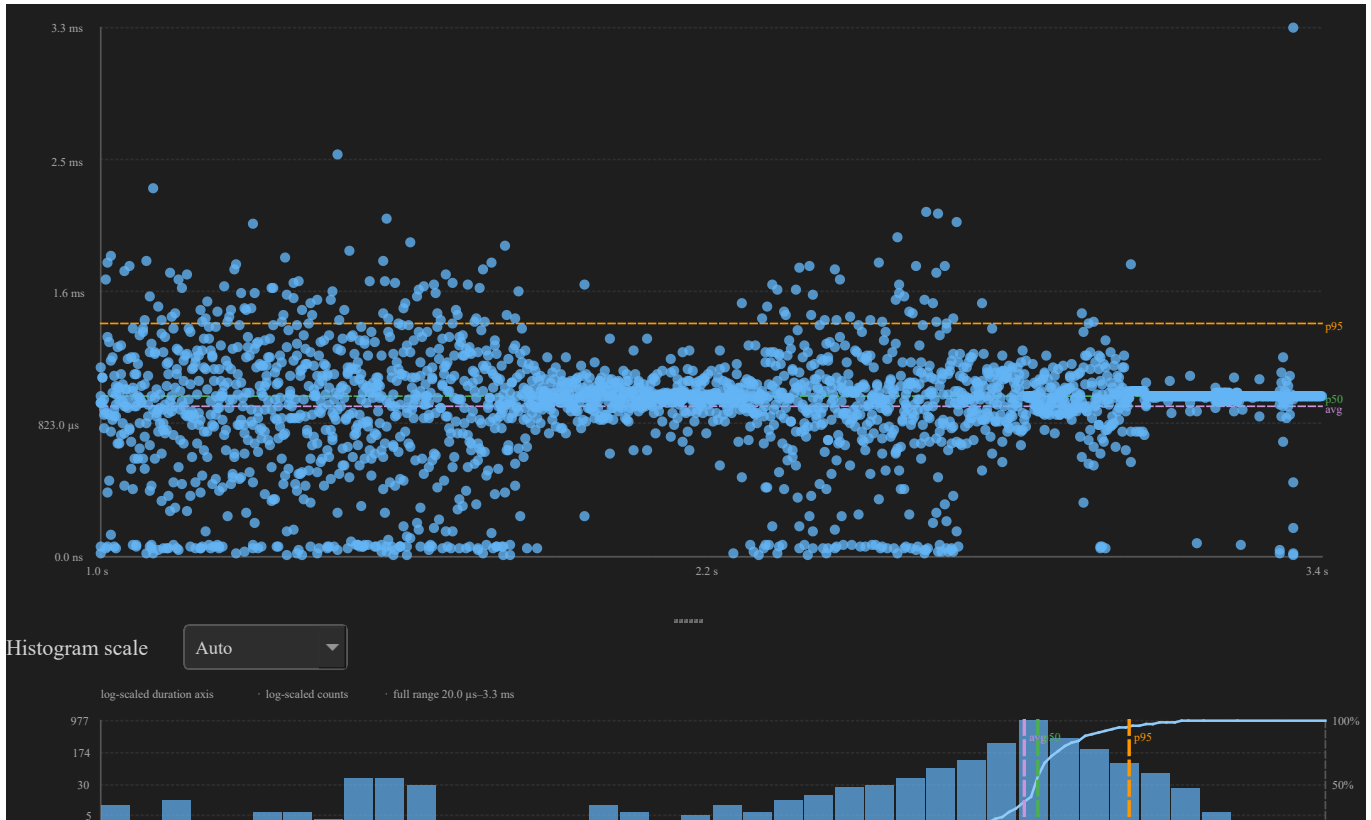
① Health

Can we trust timing?

Trace Health (TICK)

Where: Statistics → **Trace Health (TICK)** → **Tick Distribution…**

Check	Red flag	Next
Mode / CV	TICKLESS with high CV	Scope a **busy** window
Large gaps	Only idle stretches	Expected on tickless
Hard real-time	Still WARNING when busy	Compare tickful capture
Missing tick evidence	Weak timing basis	Prefer relative compare / intervals



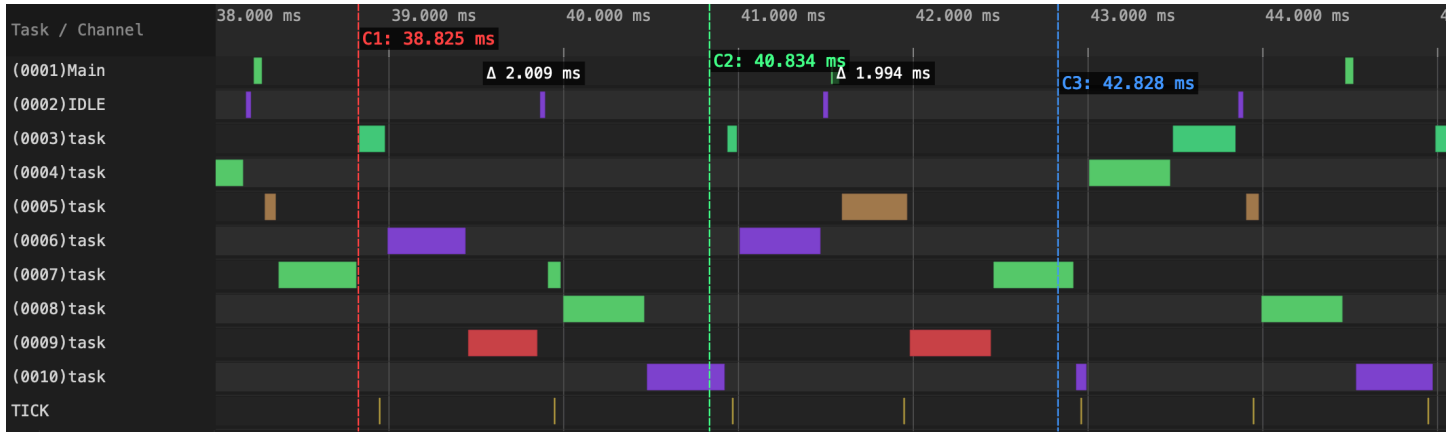
② Scope

Most bad conclusions come from mixed phases

Cursor-Scoped Analysis

Action	Effect
Click / `C`	Place cursor
`Ctrl+R`	Zoom cursor range
Limit to C1–Cn	Recompute Statistics + Analysis
Clear cursors	Return to full trace

> Heatmap / Chord follows the **visible viewport**, not the cursor-scope checkbox. Zoom first.



③ Balance

Is work placed evenly?

Core Balance

Where: Statistics → **Core Utilisation**

Signal	Meaning	Next
Score ≥ 85%, σ ≤ 30%	Generally balanced	Continue
Score < 70%	Imbalance warning	Affinity / Task × Core
σ > 30%	Spread is high	Core Time Breakdown
One core hot	Work may be pinned	Core Affinity
All high, one low	Serialization	Mutex / Preemption

Then inspect **Core Time Breakdown**, **Concurrent Core Active**, and **Kernel Switch Overhead**.

Concurrent Activity / Switch Cost

Section	Question
Core Time Breakdown	Active / Idle / Tick / Gap %
Concurrent Core Active	How often are N cores busy together?
Kernel Switch Overhead	How large are switch gaps?
Task × Core	Which task used which core?

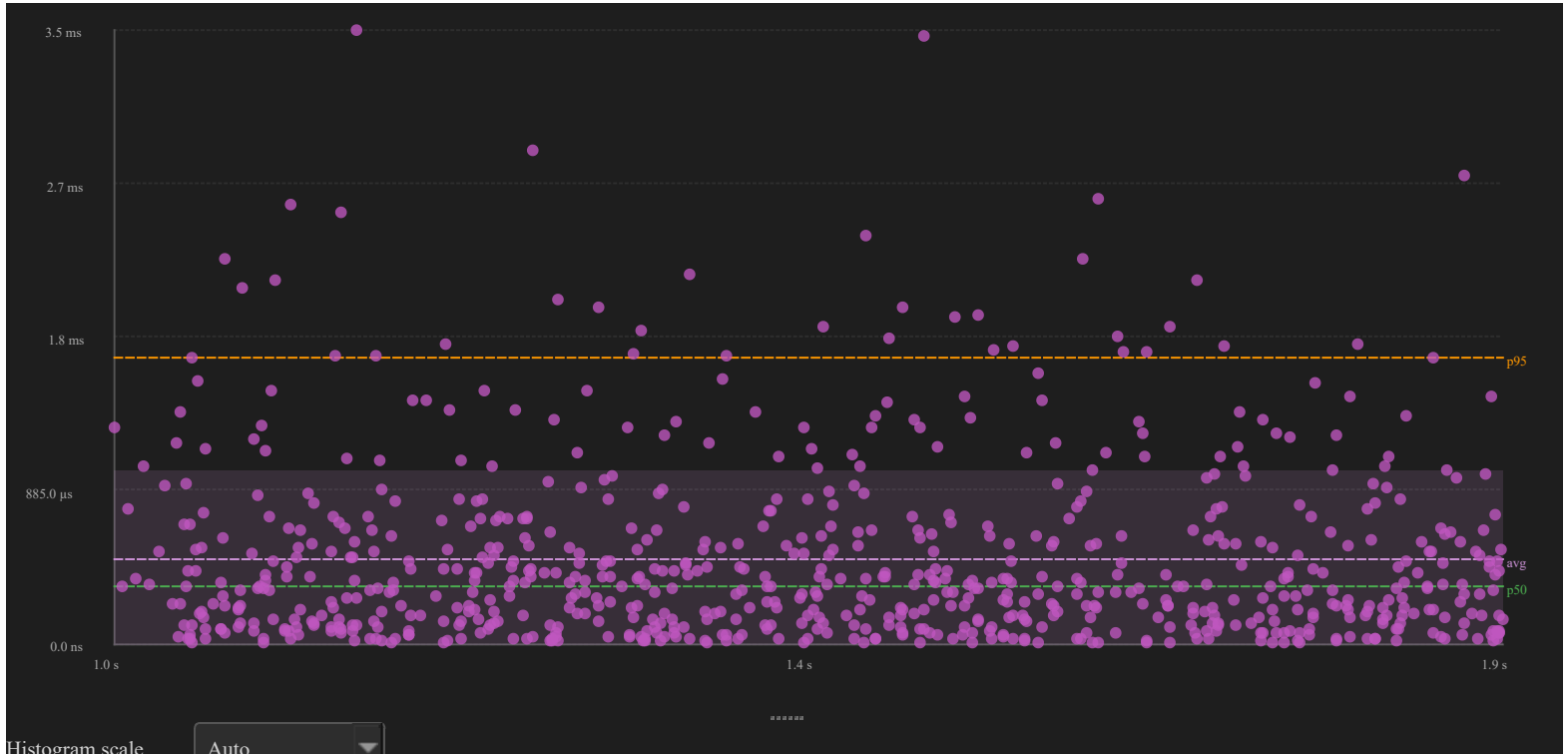
④ CPU / WCET

Which task runs too long?

Top Tasks → Execution Max

Where: Top Tasks by CPU → Execution Time Per Slice → sort **Max**

Check	Red flag	Next
Top CPU	One task dominates	Inspect task / deadline
Equal-priority workers	Fan-out / stress	Reduce concurrency / pin
Max ≫ p95	Rare spike	Click Max on timeline
Max exceeds budget	WCET issue	Set deadlines
Long slice near lock/preemptor	Interference	Mutex / Preemption



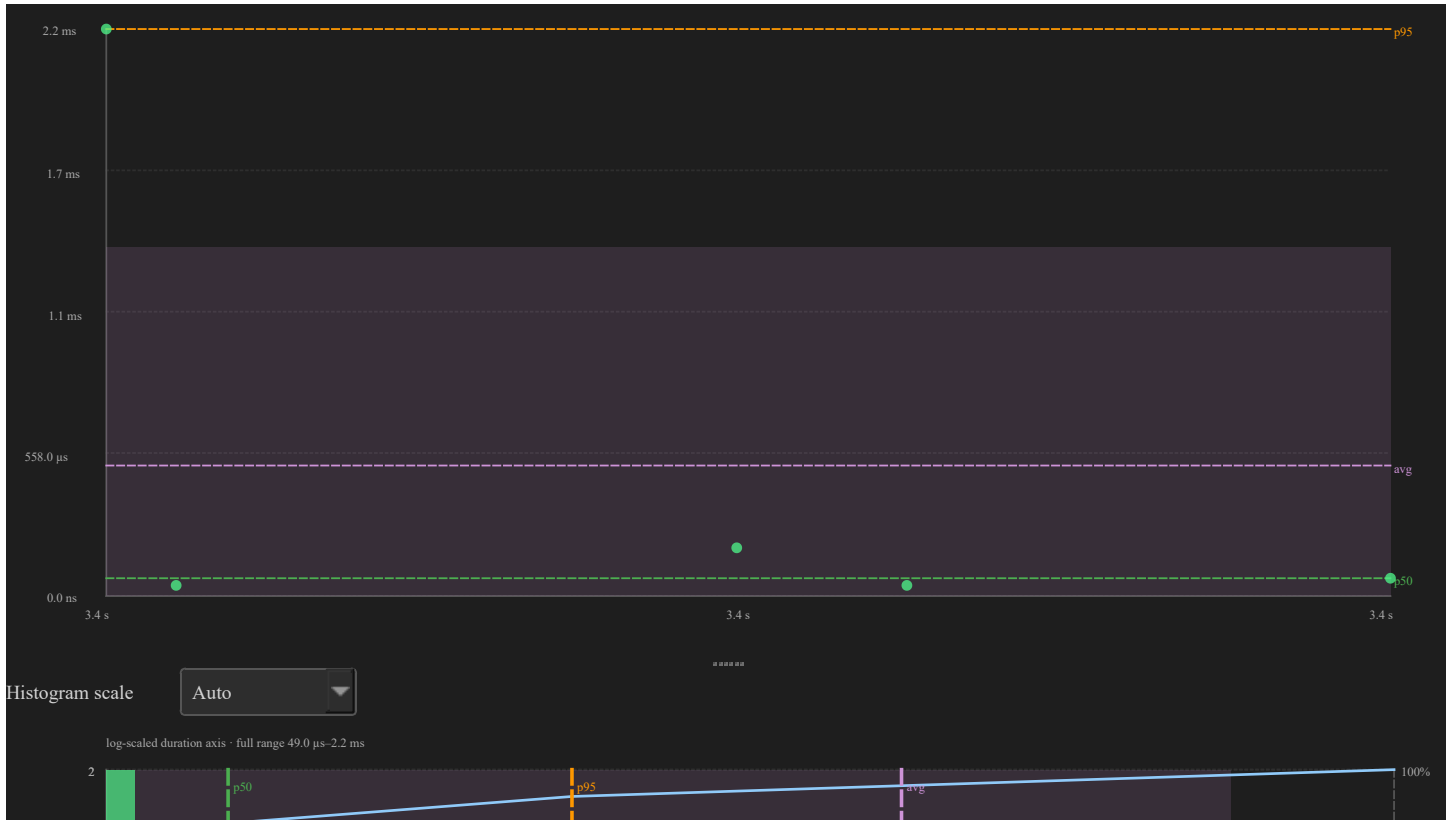
⑤ Latency

Why did the task wait?

Latency Metrics

Metric	Meaning	Use when
Blocking	Off-CPU until next run	Task waits too long
Dispatch	Ready/create/resume → run	Ready task delayed
Inter-Arrival	Start-to-start cadence	Periodic task irregular
Response	Heuristic adjacent-slice signal	Need rough tail behavior

Path: Blocking Max / p95 → Preemption Chain → Mutex / Queue → Dispatch.



⑥ Concurrency

Migrations, lock-bounce, priority inheritance

Core Migrations

Where: Statistics → **Core Migrations**

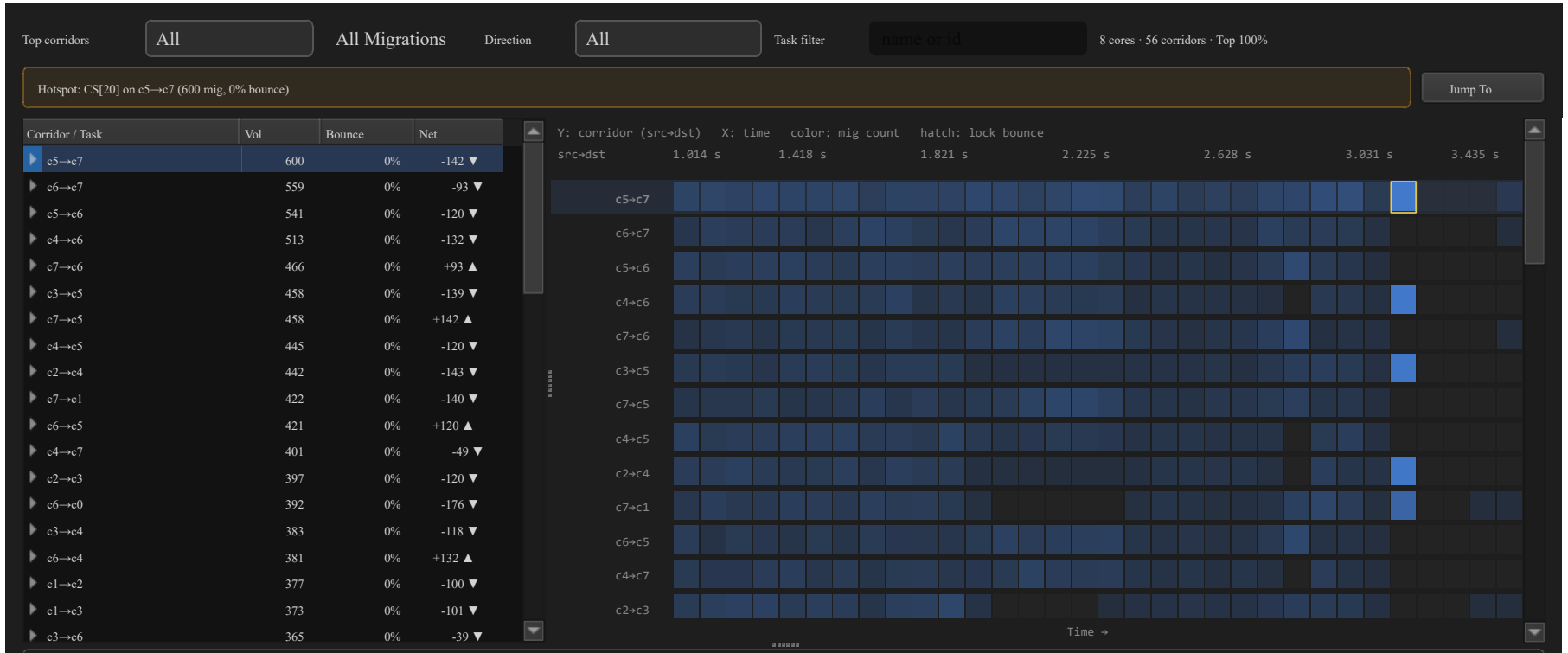
Signal	Reading
High **Rate**	Task migrates often
Short **Dwell**	Task does not stay on one core
High **Ping**	A→B→A bouncing
High **Gap after**	Migration followed by waiting
High **STI±**	Migration near STI activity

Quick path: sort Rate/Ping → click chart → lock-highlight task → enable **Load**.

Heatmap / Chord

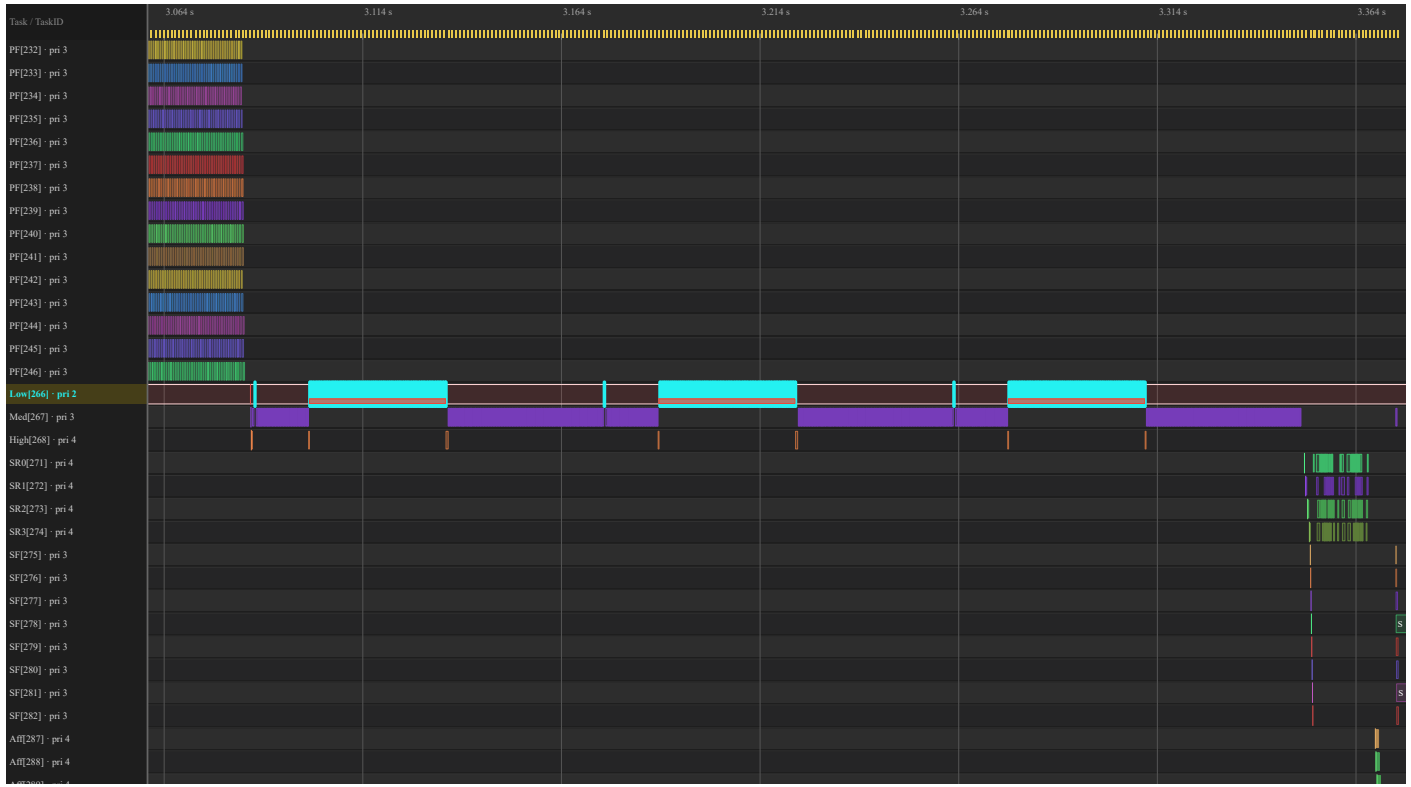
Where: toolbar **Heatmap** or **Chord**

Feature	Use
Hot cell	Migration burst in time
Ribbon	Directed core corridor
Lock Bounces Only	Mutex/queue hops only
Task filter	Name or numeric id
Inspect	Spotlight window with C1–C2



Mutex / Queue / Priority

Section	Red flag	Fix direction
Mutex / Semaphore	Issues, long holds, bounces	Shorten hold, fix pairing
Queue	Many cross-core bounces	Co-locate producer / consumer
Priority Inheritance	L/M/H pattern	Design review
Boost only	Manual / unclear boost	Verify with timeline



⑦ Compliance

Did the system do what you configured?

Pass / Fail Checks

Section	Check
Core Affinity	Observed cores ⊆ mask
Task Lifecycle	Susp/Res match; no run while suspended
Deadlines / CPU budget	Violations match real budgets
Tag Analysis	Custom values stay inside limits
Interval Analysis	Instrumented regions meet budgets

Thresholds: Settings → Display → Analysis thresholds
Example: `CS[28]=2000000` ns = 2 ms.

⑧ Compare

A fix is not done until measured again

Trace Compare

Where: open two traces → toolbar **Compare**

Page	Check
Summary	Context switches, migrations, tick, load balance
Core Util	Per-core deltas
Execution	Max / p95 / WCET movement
Blocking	Wait-time movement
Preemption	Interference movement
Sync / Mutex	Holds, issues, bounces
Response	Heuristic P99
Deadline misses	Pass/fail threshold result

Use the same workload phase on both traces.

⑨ Explain

AI after deterministic evidence

AI Assistant Flow

Use AI when:

- Statistics are scoped correctly
- Findings exist
- You clicked at least one timeline event
- You want ranking, explanation, or a report

Ask	Verify
Start Investigation / **Auto investigate**	Empty log; restart restores **Current Issue** only with a user or assistant turn
Investigate… / **Root cause…**	Apply GUI cards; click `jump:TIME`
Verify with AI…	Evidence panel result
Explain region	Times inside C1–Cn
What-if / **Optimize**	Estimate only; recapture required
Diagnostic report	Export after timeline agrees

Symptom → Metric

Symptom	Start here	Then
Unknown	**Analysis**	Named Statistics section
Tick / tickless	Trace Health	Scope busy window
Uneven SMP	Core Utilisation	Task × Core / Affinity
High switch cost	Switch Overhead	Breakdown Gap %
Slice too long	Execution Max	Preemption / Mutex
Waits too long	Blocking	Preemption / Mutex
Ready delayed	Dispatch	STI create/resume evidence
Thrashing	Migrations Rate/Ping	Heatmap / Chord
Lock-bounce	Core-Pair Bounce %	Mutex Bounces
Priority inversion	Priority Inheritance	Timeline stripe
Regression	Compare	Same phase in both traces

Checklist Card

```
☐ Full span checked
☐ Statistics + Analysis opened
☐ Trace Health checked first
☐ Relevant phase scoped with C1–Cn
☐ Finding mapped to Statistics section
☐ Max / p95 / chart point clicked
☐ Balance checked
☐ CPU / WCET checked
☐ Blocking / Dispatch checked
☐ Migrations / Mutex / Priority checked
☐ Compliance checked
☐ AI used after evidence
☐ Before / after validated with Compare
```

CI / Export Loop

```
python builds/btf_viewer.py report ../tracedata/example-8cores.btf.gz \
    --output report.html --format html

python builds/btf_viewer.py compare before.btf.gz after.btf.gz \
    --output compare.html --format html \
    --name-a "Before" --name-b "After"
```

```
Find issue → define expected improvement → change firmware/config
          → recapture → Trace Compare → export report
```

Start at the Top

Health → Scope → Measure → Verify → AI → Compare

Drill only where findings point. Scope before you escalate. AI after the timeline agrees.